

Contents

Internal Security Audit — DustSwapSweepRouter (DustSweep V3)	1
1. Executive summary	1
2. Findings summary	2
3. System overview	3
Architecture & components	3
Trust model	3
Key invariants the contract intends to preserve	4
External dependencies	4
4. Detailed findings	4
[M-1] Protocol fee is caller-controlled and trivially bypassable	4
[L-1] target and spender allowlists validated independently; arbitrary calldata to any allowlisted target	5
[L-2] Held-balance guard is absolute, not per-sweep; FoT input + stuck balance can siphon stuck tokens	6
[L-3] minAmountOut is enforced on gross (pre-fee) output; recipient can receive less than the stated minimum	6
[L-4] Single-step ownership for a broadly privileged contract	7
[L-5] Settlement transfers can revert the whole sweep (best-effort guarantee stops at settlement)	7
5. Informational & gas notes	7
Triaged tool findings (confirmed false-positives / by-design)	8
6. Methodology & tools	8
7. Recommended actions before external audit	9
8. Disclaimer	9

Internal Security Audit — DustSwapSweepRouter (DustSweep V3)

Auditor: Internal pre-audit review (Claude, Fable 5) for Akbar / DustSwap **Date:** 2026-06-13
Scope: packages/contracts/remix/DustSwapSweepRouter_Remix.sol — the DustSwapSweepRouter contract (flattened, forge flatten output) **Compiler:** solc 0.8.24+commit.e11b9ed9, EVM cancun, optimizer enabled (200 runs) **Target chain:** Base (chainId 8453) **Out of scope:** OpenZeppelin library code (Context, Ownable, Pausable, ReentrancyGuard, SafeERC20, StorageSlot), Permit2 itself, the off-chain route builder / signer, deployment scripts, the frontend.

Purpose & status. This is an internal hardening pass to run *before* engaging an external audit firm. It is **not** a substitute for that engagement. In particular, no dynamic fuzzing or invariant testing was performed (Foundry/Echidna/Medusa could not be installed in this sandbox — see §6). Treat this as a high-quality first pass that should reduce the external firm’s findings and cost, not as a clean bill of health.

1. Executive summary

DustSwapSweepRouter is a best-effort, multi-DEX “dust sweep” router: it pulls many small ERC-20 balances from a user (via Permit2 SignatureTransfer or plain allowance transferFrom), routes each through an allowlisted swap target, and settles the proceeds into a single output token (ERC-20 or native ETH) minus a protocol fee. The design is notably careful: hard per-sweep exact-amount approvals with immediate reset to zero, no standing/infinite allowances, a self-only executeRoute wrapped in try/catch for best-effort batching, snapshot-delta refund accounting that protects pre-existing balances, a reentrancy guard on the entrypoint, a pause switch, and

a MAX_FEE_BPS hard cap. The automated toolchain (Slither, Solhint, Mythril) surfaced **no exploitable issues** — every tool finding is a documented design choice or a false positive (triaged in §5.x and §6).

The substantive issues are in **economic/authorization logic**, which is exactly where automated tools are weakest:

- **1 Medium** — the protocol fee is fully **caller-controlled and bypassable**: any user can set `feeBpsOverride = 0` and pay zero fee, in both modes. This does not put user funds at risk, but it defeats the protocol's revenue mechanism.
- **5 Low** — independent (target, spender) allowlisting, an absolute (not per-sweep) held-balance check, a slippage floor applied to gross rather than net output, single-step ownership, and settlement-path DoS surfaces.
- **Several Informational / gas** notes.

No Critical or High severity issues were found. User funds are well protected by the approval model and the reentrancy guard; the headline issue is protocol-side revenue, not user-side theft. Overall risk posture: **solid foundations, with one economic correctness bug to fix and a handful of hardening items before mainnet.**

2. Findings summary

#	Title	Severity	Status
M-1	Protocol fee is caller-controlled and trivially bypassable (<code>feeBpsOverride</code> has no floor)	Medium	Open
L-1	target and spender allowlists are validated independently; arbitrary <code>calldata</code> runs against any allowlisted target under the router's identity	Low	Open
L-2	Held-balance guard uses absolute balance, not per-sweep delta → FoT input + stuck balance can siphon stuck tokens	Low	Open
L-3	<code>minAmountOut</code> slippage floor is checked on gross (pre-fee) output; recipient may receive less than the stated minimum	Low	Open

#	Title	Severity	Status
L-4	Single-step ownership (Ownable) for a contract with broad admin powers	Low	Open
L-5	A reverting feeCollector or native recipient reverts the entire sweep (best-effort guarantee does not extend to settlement)	Low	Open
I-1..I-6	Informational & gas (events, CEI ordering, centralization, coverage)	Info	Open

3. System overview

Architecture & components

A single contract, `DustSwapSweepRouter`, inheriting `OpenZeppelin Ownable`, `ReentrancyGuard`, and `Pausable`. One external entrypoint, `sweep(...)`, drives the whole flow:

1. `_resolveFeeBps / _validateParams / _validateRoutes` — input validation and fee resolution.
2. `_pullAndExecute` — snapshots unique input balances, pulls inputs (`Permit2` or `transferFrom`), runs each route best-effort via `try this.executeRoute(...)`, then refunds leftovers.
3. `executeRoute` (external, self-only) — per-leg: checks held balance $\geq$ `amountIn`, sets an exact approval to the spender (and a `Permit2 AllowanceTransfer` approval in the `Universal-Router` case), performs the arbitrary `target.call(data)`, then resets **both** approval rails to zero regardless of outcome.
4. `_settleOutput` — measures the output-token balance delta, enforces `minAmountOut`, takes the fee, and delivers net output (`ERC-20 safeTransfer`, or `WETH withdraw + native send`) to recipient.

`Permit2` EIP-712 binding is implemented via `hashRoutes + hashSweepWitness`, with `SWEEP_ROUTE_TYPEHASH`, `DUST_SWEEP_WITNESS_TYPEHASH`, and `PERMIT2_WITNESS_TYPE_STRING`.

Trust model

- **Owner** (trusted): sets `feeBps`, `feeCollector`, the `allowedTargets` and `allowedSpenders` allowlists, can pause/unpause, and can `rescueERC20/rescueNative`. The owner is highly privileged; the security of the arbitrary-call mechanism rests on the owner curating the allowlists to only well-behaved DEX routers/spenders.
- **User / caller** (untrusted): supplies routes (including arbitrary target calldata, constrained to allowlisted targets/spenders), params, and — in `Permit2` mode — a signed witness.
- **External** (untrusted): the input tokens (may be fee-on-transfer, may misbehave), the output token, and the DEX targets (mitigated by the allowlist).

Key invariants the contract intends to preserve

1. The router never grants more allowance than `route.amountIn` for any token, and never leaves a non-zero standing allowance after a leg (`set→use→reset-to-0`).
2. A user only ever pulls / refunds **their own** tokens; pre-existing balances held by the router are never returned to a sweep caller (`snapshot-delta` refunds).
3. A failed leg is skipped and its input refunded — one bad token can't brick the batch.
4. The fee never exceeds `MAX_FEE_BPS` (3%).
5. sweep is non-reentrant and respects the pause switch.

Invariants 1–5 hold in the reviewed code. Invariant 2 has a narrow exception when stuck balances exist (see **L-2**). The fee mechanism is *capped* (invariant 4 holds) but not *floored* (see **M-1**).

External dependencies

Permit2 (canonical, immutable address), WETH9 on Base, and arbitrary allowlisted DEX routers. SafeERC20 is used for all protocol-controlled transfers; `raw .call` is used deliberately for the swap leg and for best-effort refunds.

4. Detailed findings

[M-1] Protocol fee is caller-controlled and trivially bypassable

- **Severity:** Medium — Impact: Medium (direct, ongoing loss of protocol fee revenue; no user-fund risk), Likelihood: High (any caller, no preconditions, one parameter). Per an impact×likelihood matrix this lands at Medium/High; rated **Medium** because no user or protocol *principal* is at risk — only fee income. **If fee revenue is core to DustSwap's sustainability, treat this as High.**
- **Location:** `SweepParams.feeBpsOverride` (line 1073); `_resolveFeeBps` (lines 1298–1301); used at 1216 and bound into the witness at 1377; fee charged at 1493–1494.
- **Description.** The effective fee is taken directly from the caller-supplied `params.feeBpsOverride`:

```
function _resolveFeeBps(uint16 feeBpsOverride) internal view returns (uint16 effectiveFeeBps)
    effectiveFeeBps = feeBpsOverride == FEE_OVERRIDE_SENTINEL ? feeBps : feeBpsOverride;
    if (effectiveFeeBps > MAX_FEE_BPS) revert FeeTooHigh(); // upper bound only — no floor
}
```

There is an **upper** bound (`MAX_FEE_BPS`) but no **lower** bound and no authorization that the override matches a protocol-mandated value. Any caller can pass `feeBpsOverride = 0` (or any value below the intended fee) and the contract happily charges that.

This is not fixed by Permit2 mode. In Permit2 mode the witness binds `effectiveFeeBps` — but it binds *the value the caller themselves chose*. A user crafting their own routes/witness simply signs `feeBpsOverride = 0`. In Allowance mode there is no signature at all, so the override is even more obviously free.

- **Impact / scenario.** A power user (or a competing frontend, or a wrapper contract) calls sweep directly with `params.feeBpsOverride = 0`. The swap proceeds normally, `feeAmount = grossAmountOut * 0 / 10_000 = 0`, and the full gross output is delivered to the recipient. The protocol collects nothing. Repeated across all sophisticated users, fee revenue trends to zero while the honest frontend's users (who send the sentinel) still pay. Economic exploit, atomic, repeatable, zero cost.
- **Recommendation.** Decide who is allowed to *lower* the fee, and enforce it on-chain. Options, smallest first:

1. **Remove the override entirely** and always use the storage feeBps (effectiveFeeBps = feeBps). Simplest; the contract fee is the contract fee.
 2. **Floor the override:** if (effectiveFeeBps < feeBps) revert FeeTooLow(); so an override can only ever *raise* the fee (e.g. promotional/partner sweeps), never lower it.
 3. If per-sweep discounts are a real product requirement, gate them: only honor an override below feeBps when accompanied by an **owner/relayer signature** over (user, feeBpsOverride, deadline), so discounts are protocol-authorized rather than self-served.
- **Status:** Open.
-

[L-1] target and spender allowlists validated independently; arbitrary calldata to any allowlisted target

- **Severity:** Low — Impact: Medium-to-High *if* a dangerous target/spender is ever allowlisted, Likelihood: Low (requires owner to allowlist an exploitable contract; the model explicitly trusts the owner to curate). Net **Low**, but it is the single most important thing to get right operationally.
 - **Location:** `_validateRoutes` (1314-1323) checks `allowedTargets[route.target]` and `allowedSpenders[route.spender]` *separately*; `executeRoute` performs `route.target.call(route.data)` (1278) with fully attacker-controlled data.
 - **Description.** Two independent boolean maps mean any allowlisted target may be paired with any allowlisted spender, and **arbitrary calldata** is executed against that target *with the router as msg.sender*. The contract's safety therefore reduces entirely to: "every allowlisted target is safe to call with arbitrary data, in any combination with every allowlisted spender, while holding an exact-amount approval." That is a strong, easy-to-violate assumption. Examples of how curation can go wrong:
 - Allowlisting a router that exposes a generic `multicall`, `sweepToken`, `pull`, or `unwrapWETH9`-style helper that operates on tokens/ETH the DustSwap router holds or has approved.
 - Allowlisting a token/contract as a target that, combined with the granted approval to a *different* allowlisted spender, lets the swap move funds in an unintended way.
 - Accidentally allowlisting the router's own address, an EOA, or `Permit2` as a target.
 - **Impact / scenario.** No exploit exists against a correctly curated allowlist (DEX targets that only swap). The risk is realized through operational error: a single mis-allowlisted contract could let a crafted `route.data` move the in-flight amountIn (or, with L-2, more) to an attacker-chosen destination. Because the approval is exact and reset each leg, blast radius is bounded to one leg's amountIn per call, but that is still user-fund loss for the active sweep.
 - **Recommendation.**
 - Prefer **whitelisting (target, spender) pairs** (e.g. `mapping(address target => mapping(address spender => bool))`) rather than two independent sets, so only intended combinations are valid.
 - Optionally **constrain the function selector** per target (allowlist the 4-byte selector(s) you expect, reject everything else) instead of accepting arbitrary data.
 - Document and enforce an allowlisting runbook: never allowlist contracts with `multicall`/arbitrary-execution surfaces, the router itself, EOAs, or token contracts.
 - **Status:** Open.
-

[L-2] Held-balance guard is absolute, not per-sweep; FoT input + stuck balance can siphon stuck tokens

- **Severity:** Low — Impact: Low-Medium (bounded leakage of tokens already considered “stuck/awaiting rescue”), Likelihood: Low (requires the router to be holding a pre-existing balance of the same token the attacker sweeps as fee-on-transfer).
 - **Location:** `executeRoute` held-balance check (1263): `if (tokenIn.balanceOf(address(this)) < route.amountIn) revert InsufficientHeldBalance();` vs. the snapshot-delta model in `_snapshotInputs` (1419-1457) / `_refundLeftoverInputs` (1461-1484).
 - **Description.** The fee-on-transfer protection (“never approve more than we actually received”) checks the router’s **total** balance of `tokenIn`, not the amount *this sweep* brought in. If the router holds a pre-existing balance P of `tokenX` (stuck tokens awaiting `rescueERC20`), and a user sweeps `tokenX` with $\text{amountIn} = A$ where `tokenX` is fee-on-transfer so only A' $< A$ actually arrives, then $\text{balanceOf} == P + A'$. If $P \geq A - A'$, the guard passes, the leg approves and swaps the full A , consuming $A - A'$ of the pre-existing P . The swap output is credited to *this caller’s* gross. Afterwards $\text{balanceOf}(\text{tokenX}) = P - (A - A') < P$, so `current > initialBalance` is false and no over-refund is detected — the difference is silently converted into the attacker’s output.
 - **Impact / scenario.** A user who notices stuck `tokenX` in the router (e.g. a prior failed refund, `RefundFailed` event) sweeps `tokenX` as a FoT input to convert up to the stuck amount into output they receive, bypassing the owner’s `rescueERC20`. Bounded by the stuck balance and the FoT fee fraction; only possible when stuck balances exist (an exceptional state by design).
 - **Recommendation.** Make the held-balance guard consistent with the snapshot-delta model: compare against what *this sweep* delivered. E.g. capture each input token’s balance just before its `transferFrom/Permit2` pull and pass the *delta actually received* into `executeRoute`, approving/swapping `min(amountIn, received)` rather than checking total balance. Alternatively, route all swaps off the per-token *received* amount rather than the requested `amountIn`.
 - **Status:** Open.
-

[L-3] `minAmountOut` is enforced on gross (pre-fee) output; recipient can receive less than the stated minimum

- **Severity:** Low — Impact: Low (user receives slightly less than the number they set as their floor; no theft), Likelihood: High (every sweep with a non-zero fee).
- **Location:** `_settleOutput` (1490-1494): `grossAmountOut` is checked against `minAmountOut`, then the fee is subtracted to produce `netAmountOut` which is what the recipient actually gets.

```
grossAmountOut = balanceOf(actualOutput) - st.initialOutputBalance;  
if (grossAmountOut < params.minAmountOut) revert InsufficientOutput(); // floor on GROSS  
feeAmount      = (grossAmountOut * st.effectiveFeeBps) / BPS_DENOMINATOR;  
netAmountOut = grossAmountOut - feeAmount;                          // recipient gets NET
```
- **Description / impact.** A user who sets `minAmountOut` as “the least I’m willing to receive” will actually receive `minAmountOut - fee` in the worst case. The slippage guarantee is off by the fee. This is a correctness/UX issue rather than a security hole, but it can surprise integrators and weakens the protection the user thinks they have.
- **Recommendation.** Either (a) check the floor on the delivered amount: `if (netAmountOut < params.minAmountOut) revert InsufficientOutput();`, or (b) clearly document that `minAmountOut` is a *gross* floor and have the frontend/signer set it accordingly. Pick one and make the witness/UX consistent with it.

- **Status:** Open.
-

[L-4] Single-step ownership for a broadly privileged contract

- **Severity:** Low — Impact: High if it happens (admin functions become permanently un-callable), Likelihood: Low (requires an owner mistake). Net **Low**.
 - **Location:** contract DustSwapSweepRouter is Ownable, ... (1039); OZ Ownable.transferOwnership (379-384).
 - **Description.** The owner controls fees, the allowlists (the core security control, per L-1), pause, and rescue. Plain Ownable transfers ownership in a single step — a transfer to a wrong/uncontrolled address immediately and irreversibly hands over (or bricks) all admin power.
 - **Recommendation.** Use OpenZeppelin Ownable2Step (propose + accept), and use a multisig / timelock as the owner in production. Given the power of setAllowedTarget/setAllowedSpender and rescueERC20, a timelock on at least the allowlist and rescue setters is worth considering.
 - **Status:** Open.
-

[L-5] Settlement transfers can revert the whole sweep (best-effort guarantee stops at settlement)

- **Severity:** Low — Impact: Medium (temporary denial of service for a given output token / recipient), Likelihood: Low.
 - **Location:** _settleOutput fee transfer (1499), native send (1505-1506), ERC-20 delivery (1508).
 - **Description.** The route loop is carefully best-effort, but settlement is all-or-nothing. If feeCollector is blacklisted by the output token (e.g. USDC blacklist), or the output token reverts on transfer to the collector, **every** sweep into that output token reverts. If the native recipient rejects ETH, that sweep reverts. The first case is owner-influenced but token-dependent; it can silently DoS an entire output market.
 - **Impact / scenario.** An output token that blacklists the configured feeCollector makes all sweeps to that token fail until the owner changes the collector. Not a fund loss, but an availability and reliability issue.
 - **Recommendation.** Consider a pull-payment pattern for fees (accrue per-token, let the collector withdraw) so a collector/token problem can't block users; and/or wrap the native send so a failed recipient transfer falls back to WETH delivery rather than reverting. At minimum, document the operational requirement that feeCollector must be transfer-compatible with all supported output tokens.
 - **Status:** Open.
-

5. Informational & gas notes

- **I-1 — Events emitted after external calls (CEI ordering).** _settleOutput emits DustSwept/ProtocolFeePaid after token/native transfers. Benign here (the whole flow is nonReentrant), but tightening to checks-effects-interactions is good hygiene.
- **I-2 — Unindexed address parameters in events.** FeeCollectorUpdated, Paused/Unpaused, and several others have non-indexed address/amount params (flagged by Slither/Solhint). Indexing key addresses improves off-chain filtering.

- **I-3 — Centralization.** rescueERC20/rescueNative let the owner move any non-in-flight balance; combined with allowlist and fee control this is significant trust. Mitigated by atomicity (rescue can't run mid-sweep) and by the snapshot-delta refund model, but should be disclosed to users. Pair with L-4 (multisig/timelock).
- **I-4 — minAmountOut / fee semantics** — see L-3; documenting the gross-vs-net convention removes ambiguity for integrators.
- **I-5 — Gas (low priority):** struct field packing (SweepParams, SweepState), and $\geq$ / $>$ strict-inequality micro-optimizations flagged by Solhint. Cosmetic; do not chase these at the expense of clarity in a security-sensitive contract.
- **I-6 — Coverage gap.** No fuzzing/invariant testing or fork testing was performed in this pass (see §6). The invariants in §3 are strong candidates for Foundry invariant tests and Echidna properties before mainnet.

Triaged tool findings (confirmed false-positives / by-design)

Tool finding	Disposition
Slither arbitrary-send-eth in _settleOutput	By design — ETH is sent to the user-specified recipient; that is the function's purpose, and it is nonReentrant.
Slither uninitialized-local (SweepState st)	False positive — struct is zero-initialized then every used field is set before read.
Slither incorrect-equality in _refundLeftoverInputs	By design — the <code>ret.length == 0</code> check is the intentional SafeERC20-style handling of non-compliant return data; the comment documents why short returns are not decoded.
Slither calls-in-loop (snapshot/execute/refund)	Acceptable — loop is bounded by <code>MAX_BATCH_SIZE = 50</code> ; the route loop is intentionally best-effort try/catch.
Slither assembly usage	From OZ / standard — <code>_snapshotInputs</code> array-trim and the OZ libraries; memory-safe.
Slither/Solhint timestamp comparison	Acceptable — used only for the deadline check, standard and not a randomness/ordering dependency.
Solhint avoid-low-level-calls	By design — <code>raw .call</code> is required for the arbitrary swap leg and for fault-tolerant refunds.
Mythril (EtherThief, ArbitraryStorage, ArbitraryDelegateCall, UncheckedRetval)	No issues detected.

6. Methodology & tools

- **Static analysis:** Slither 0.11.5 (101 detectors), Solhint 6.2.1 (security profile).
- **Symbolic execution:** Mythril 0.24.8, bytecode mode (EtherThief / arbitrary-storage / arbitrary-delegatecall / unchecked-retval modules).
- **Comprehension:** Surya 0.4.13 (function/visibility map, inheritance).
- **Compiler:** solc 0.8.24+commit.e11b9ed9 (exact match to the contract's Remix/BaseScan verify settings).
- **Manual review:** function-by-function against an auditor checklist covering access control, reentrancy (single-/cross-function, read-only), arithmetic/accounting, oracle/MEV, token quirks (fee-on-transfer, missing return values), approval hygiene, DoS/griefing, and economic logic.

Not covered (do before / leave to the external firm): - **Dynamic testing** — no Foundry invariant tests, Echidna/Medusa fuzzing, or Halmos symbolic proofs. The sandbox blocks the binary CDNs/Go-Rust toolchains those need. Run them on an open machine or CI against the §3 invariants. This is the most important gap. - **Mainnet fork testing** against real Permit2, real WETH on Base, and the actual DEX targets you intend to allowlist (this is where target-safety / L-1 should be empirically validated). - Off-chain components: the route builder, the EIP-712 signer, and the relayer. - Economic modeling of the fee under adversarial routing.

7. Recommended actions before external audit

1. **Fix M-1** — decide the fee-override policy and enforce a floor (or remove the override). This is the one logic bug that materially changes contract behavior.
 2. **Harden L-1** — move to (target, spender) pair allowlisting and/or per-target selector allowlisting; write the allowlisting runbook.
 3. **Address L-2/L-3** — make held-balance and slippage checks delta/net-consistent.
 4. **Adopt Ownable2Step + multisig/timelock** (L-4) and document centralization (I-3).
 5. **Write the invariant/fuzz suite** (§3 invariants) and **fork-test** against the real allowlisted targets before the external engagement — it will sharply reduce their findings and your cost.
-

8. Disclaimer

This internal review reflects the state of the code at the reviewed file and is not a guarantee of the absence of vulnerabilities. It did not include dynamic fuzzing, invariant testing, or fork testing, and does not replace a full external audit. Security is an ongoing process: re-review after any change, complete the dynamic-analysis gap noted in §6, engage the external firm, and operate with a timelock/multisig and a bug bounty in production.